Guía rápida para exponer VetSystem

Oct 30, 2026 · @Fernando

En una frase: VetSystem es un sistema veterinario hecho en **Django** (el núcleo) con **4 microservicios en FastAPI**, base de datos **MySQL** y **Redis**, todo detrás de un **Nginx con HTTPS** para usarlo desde cualquier PC de la red.

1. Con qué está hecho (las 5 capas del diagrama)

Capa	Tecnología	Para qué
1. Cliente	HTML5, Bootstrap 5, JavaScript (Fetch), Chart.js, FontAwesome	Pantallas y gráficas en el navegador
2. Red y proxy	**Nginx** en Docker, HTTPS (certificado propio), nip.io como DNS	Única puerta de entrada: cifra y reparte cada petición
3. Backend	**Django 6.1** (Python), patrón MVT, 11 apps	Lógica del negocio, usuarios, login con Google
4. Microservicios	**FastAPI** en puertos 8081–8084	Notificaciones, Facturación, Telemedicina, Reportes
5. Datos	**MySQL 8.4** y **Redis 7** en Docker	MySQL guarda todo; Redis guarda sesiones y caché

2. Cómo funciona

1. El navegador pide una página por **HTTPS** (puerto 443).
2. **Nginx** la recibe: si es `/sala/` o `/ws/` la manda a Telemedicina; todo lo demás va a **Django**.
3. Django revisa sesión y permisos, consulta **MySQL** con su ORM (sin escribir SQL) y arma la página con su plantilla.
4. Cuando necesita algo especial, Django llama a un **microservicio por HTTP con JSON** (API REST): firmar la factura, preparar el WhatsApp, crear la sala de video o generar el PDF.

5. La respuesta regresa por Nginx al navegador.

Idea clave: cada microservicio es un programa aparte con su propio puerto. Si uno se apaga, Django **no se cae** (las llamadas tienen tiempo límite); la factura o la cita se guardan igual y se muestra un aviso.

3. Videollamada (lo que seguro te preguntan)

Usa **WebRTC**, la tecnología de video que traen los navegadores. El video **viaja directo de una PC a la otra** (peer-to-peer); el servidor solo las presenta.

Con qué: microservicio de Telemedicina en **FastAPI (puerto 8083)** + **WebSocket** + WebRTC en el navegador + servidor **STUN de Google**.

1. Al crear una cita con *teleconsulta*, Django le pide al microservicio crear una **sala** (ej. `vet-23-a1b2c3`).
2. El veterinario y el cliente abren la sala; el navegador pide permiso de **cámara y micrófono** (`getUserMedia`).
3. Cada navegador se conecta al **WebSocket** `/ws/sala/<id>`. Ese canal es la **señalización**: el servidor solo reenvía mensajes de uno al otro.
4. Por ahí intercambian: `join` (llegué) → `offer` (mi propuesta de video) → `answer` (acepto) → `ice` (las rutas de red para encontrarse).
5. El **STUN** le dice a cada PC cuál es su dirección de red. Con eso se conectan directo y el video empieza.
6. Al colgar se envía `leave` y la otra pantalla se libera.

Por qué HTTPS: el navegador **solo permite la cámara en sitios seguros**. Por eso pusimos Nginx con certificado.

Extra: hay un selector de cámara que evita la cámara virtual de OBS.

4. Chat interno

El chat usa **el mismo WebSocket de la sala**, así que no necesita otro servidor.

1. Al escribir y presionar enviar, el navegador manda `{ chat: "texto", usuario: "nombre" }` por el WebSocket.
2. El microservicio (FastAPI) lo **reenvía a los demás de esa sala** (`broadcast_to_room`), nunca a otras salas.
3. El otro navegador lo recibe al instante y lo muestra en la caja de mensajes.

Es tiempo real porque el WebSocket es una conexión que se queda abierta (no hay que recargar). **Va cifrado** porque viaja por `wss://` (WebSocket sobre HTTPS). Los mensajes no se guardan: duran lo que dura la sesión.

5. Mensajes por WhatsApp

Los genera el microservicio de **Notificaciones (FastAPI, puerto 8081)** usando los **enlaces oficiales de WhatsApp** (`wa.me`).

1. Al crear una cita o una vacuna, Django envía al microservicio los datos: cliente, teléfono, mascota, fecha y hora.
2. El microservicio **arma el mensaje** (ej. *"Hola Andrea, le recordamos la cita de Bruno el 30/10 a las 08:30…"*).
3. Limpia el teléfono y le agrega el **código de Guatemala 502** si tiene 8 dígitos.
4. Devuelve un enlace como `https://wa.me/50255123456?text=Hola...` (el texto va codificado en la URL).
5. En pantalla aparece el botón verde **Enviar por WhatsApp**: abre WhatsApp Web o la app con el chat y el mensaje ya escritos, y el personal solo presiona Enviar.

El mensaje sale desde la cuenta de WhatsApp abierta en esa PC (la de la clínica). También queda en el historial del panel de microservicios.

Si preguntan por qué no se envía solo: el envío automático exige la **API de WhatsApp Business** de Meta o Twilio, que es de pago y pide verificar la empresa. El microservicio ya hace todo lo demás; solo cambiaría ese último paso.

6. Otra PC y respuestas rápidas

Otra PC en la misma red entra con `https://<IP-del-servidor>.nip.io/login/` (hoy `192.168.3.41`). Funciona porque Django y los microservicios escuchan en `0.0.0.0` (toda la red), el firewall abre los puertos 80 y 443, y **nip.io** convierte el nombre en la IP (Google no acepta IPs para el login).

Si preguntan…	Responde
¿Monolito o microservicios?	Híbrido: núcleo Django + 4 microservicios independientes que se hablan por API REST

Si preguntan…	Responde
¿Qué pasa si cae un microservicio?	Django sigue funcionando; las llamadas tienen tiempo límite y muestran un aviso
¿Para qué Nginx?	Proxy inverso: pone el HTTPS y reparte las rutas a Django o a cada microservicio
¿La factura es válida?	Simula la FEL: hash SHA-256, firma, QR y estado ACEPTADO; en producción se conecta a un certificador real
¿El pago con tarjeta?	Simulado como Stripe: devuelve un código de autorización que queda en la factura
¿Por qué FastAPI?	Liviano, soporta WebSocket y genera documentación automática en `/docs`
¿Login con Google?	OAuth 2.0 con django-allauth; nunca vemos la contraseña, solo nombre y correo
¿Diferencia con el diagrama?	El diagrama pone colas en Redis; usamos llamadas REST directas, más simple y demostrable

Orden para la demo: Panel de microservicios (4/4 ONLINE) → cita con teleconsulta y botón de WhatsApp → videollamada y chat con la otra PC → factura con QR y PDF.